Team-based Final Project

1 Instructions

This is a *team* project. A team can be comprised of 1-3 members; teams of 4+ should split into smaller teams. You are allowed to use books, notes, and online references. In principle the use of AI is allowed, but you are encouraged to try to do all of the coding by hand for extra practice. If two distinct teams submit solutions which are suspiciously similar, both will receive a zero, and this could happen if both teams use the same AI (so, another reason to limit its use). We will make every effort to be generous and fair with partial credit, so if you are unsure of how to complete a problem, you maximize the expected points simply by doing as much as you can.

Completed projects must be submitted on the web, in the same manner as for homework submissions. As in the rest of the course, your team can choose your language from among python, java, and c++. If you use python, all floating point and matrix calculations must use numpy. If you choose java or c++, you should also use the project structure (e.g. maven, cmake) and the linear algebra libraries (EJML, Eigen) that were used in all of our in-class coding examples.

If a notebook is used (such as `.ipynb` format), you must submit both the notebook file, and a PDF export. The PDF file must contain all code, comments and diagrams that were available in the notebook. If you don't use notebooks, then submit source code as a zipped-up project directory ready to compile (like the examples provided by the professor in some modules), along with a human-readable PDF summarizing your results with all plots. Code should be well-documented via comments. Every function should come with a comment explaining what it does, and what the inputs and outputs are supposed to be.

Each problem will be graded independently – here is what we mean by that. For example, if you were unable to complete the line search problem in a satisfactory manner, you can still complete the BFGS problem using a pre-packaged line search method such as `scipy.optimize.line_search`. In this case, a team's project could in principle receive full points on the BFGS problem even if they receive zero points on the line search problem.

2 Line Search

Problem 2.1. Implement a line search method that targets the strong Wolfe conditions (also called Armijo and strong Wolfe curvature conditions). Specifically, you will need to implement Algorithms 3.2 and 3.3, pp. 59–60 of Wright and Nocedal (1999), *Numerical Optimization*.

Notes:

1. See the accompanying file for Chapter 3 of Wright and Nocedal (1999), where you can read about Algorithms 3.2 and 3.3 with accompanying background. You may want to read the entire chapter before starting this problem.

3 Quasi-Newton Optimizer

Recall from class that the BFGS method is one of the most important quasi-Newton methods. It maintains a symmetric positive definite matrix J_k that approximates the inverse Hessian $\nabla^2 f(x_k)^{-1}$, and computes the search direction

$$d_k = -J_k \nabla f(x_k).$$

The next iterate is then obtained by a line search.

Recall also that the curvature condition implies that the search direction remains a descent direction. The curvature condition is typically enforced by choosing the step length using a line search satisfying the Wolfe conditions.

The user may, or may not, have a formula for the analytic gradient. Your code should work in either case. If no analytic gradient is provided by the user, your code should compute the gradient numerically using the four-point centered difference approximation discussed in class.

If you chose python, then the function that computes the gradient can be a callable, optional argument to the main optimization method. If you chose java, there are multiple ways to achieve the desired behavior. You could mimic python and use the `ToDoubleFunction<double[]>` interface for the objective, and `Optional<Function<double[], double[]>>` or something similar to represent the (optional) gradient. More elegant and possibly more efficient would be to define a single interface representing a differentiable objective:

```
public interface ObjectiveFunction {

    double f(double[] x);

    // over-writes the contents of array "g" with the gradient g := \nabla f(x)
    // only called if hasAnalyticGradient() returns true, otherwise finite-difference methods are used
    void grad(double[] x, double[] g);

    boolean hasAnalyticGradient();
}
```

Problem 3.1. Implement the BFGS method for nonlinear optimization as discussed in class, with line search performed by the method that you implemented in the previous problem. As discussed above, if no analytic gradient method is provided, compute it numerically using (fourth order) finite-difference methods.

Problem 3.2. Test your BFGS implementation on a few test problems for which you know the exact solution. In your tests, also plot the function value as a function of the iteration number to illustrate progress.

Notes:

1. We did several examples in lecture which could be test problems, such as the ill-conditioned quadratic example. There is also the famous Rosenbrock function which is often used to diagnose problems in optimization software.

2. You are encouraged to come up with your own test problems, but if you are having trouble thinking of good test problems, there is a website with many examples:
   `https://www.sfu.ca/~ssurjano/optimization.html`

4 Runge–Kutta Method

Our goal is to approximate the solution of the problem

$$\mathbf{y}' = \mathbf{f}(t, \mathbf{y}), \quad t \geq t_0, \quad \mathbf{y}(t_0) = \mathbf{y}_0. \tag{4.1}$$

Here prime denotes a time derivative, $\mathbf{f}$ is a sufficiently well-behaved function that maps

$$[t_0, \infty) \times \mathbb{R}^d \longrightarrow \mathbb{R}^d$$

and the initial condition $\mathbf{y}_0 \in \mathbb{R}^d$ is a given vector.

As a recap of lecture, implicit Runge Kutta Methods are called "implicit" because the stages are given in terms of coupled equations they must satisfy. For the $\nu = 2$ method, the update is

$$y_{n+1} = y_n + h\,(b_1\xi_1 + b_2\xi_2). \tag{4.2}$$

where we need to solve

$$\xi_1 = f(t_n + c_1\,h,\ y_n + h\,(a_{1,1}\xi_1 + a_{1,2}\xi_2))\,, \tag{4.3}$$

$$\xi_2 = f(t_n + c_2\,h,\ y_n + h\,(a_{2,1}\xi_1 + a_{2,2}\xi_2))\,. \tag{4.4}$$

When $\mathbf{f}$ is nonlinear, then (4.3)–(4.4) represent a system of coupled nonlinear equations which, in applications, are to be solved numerically.

Problem 4.1. Implement the two-stage implicit Runge Kutta Method as discussed in class for the general initial value problem (4.1). Your implementation of the update (4.2) should use the BFGS solver to numerically solve (4.3)–(4.4).

Consider as an example, the forced damped Duffing oscillator

$$mx'' + cx' + kx + \beta x^3 = F\cos(\Omega t),$$

which is naturally equivalent to the first-order system

$$\begin{aligned} x'(t) &= v(t), \\ v'(t) &= \frac{1}{m}\left(F\cos(\Omega t) - cv(t) - kx(t) - \beta x(t)^3\right). \end{aligned}$$

This takes the form (4.1) with $\mathbf{y}(t) = (x(t), v(t))$.

Problem 4.2. Test your Runge–Kutta solver on the first-order IVP version of the Duffing oscillator. Take

$$m = 1, \quad c = 0.1, \quad k = 1, \quad \beta = 5, \quad F = 0.3, \quad \Omega = 1.2.$$

with initial data $x(0) = 0.2,\ v(0) = 0$. The *phase portrait* is defined to be the 2d parametric curve formed by $(x(t), v(t))$. Plot the phase portrait for $t = 0$ to 50 and report your values for $(x(t), v(t))$ at $t = 50$.